Đặc tả Yêu cầu Hệ thống và Chiến lược Triển khai Hạ tầng Dữ liệu Real-time: Tích hợp Debezium, Kafka, Apache Flink và ClickHouse

Sự bùng nổ của nền kinh tế số và yêu cầu về tính tức thời trong việc ra quyết định đã thúc đẩy các doanh nghiệp chuyển dịch từ kiến trúc xử lý dữ liệu theo lô (batch processing) sang xử lý luồng thời gian thực (real-time stream processing).[1] Trong bối cảnh này, việc xây dựng một hệ thống có khả năng thu thập, xử lý và phân tích dữ liệu ngay khi sự kiện phát sinh không chỉ là lợi thế cạnh tranh mà còn là yếu tố sống còn để tối ưu hóa vận hành, phát hiện gian lận và nâng cao trải nghiệm khách hàng.[1] Báo cáo này trình bày một cái nhìn toàn diện và chuyên sâu về kiến trúc hệ thống dữ liệu hiện đại, kết hợp các công nghệ hàng đầu như Change Data Capture (CDC), Message Broker phân tán, Stream Processing và cơ sở dữ liệu OLAP hiệu năng cao.

Phân tích Bối cảnh và Tầm nhìn Kiến trúc

Kiến trúc dữ liệu truyền thống thường dựa trên các quy trình ETL (Extract, Transform, Load) chạy định kỳ, dẫn đến độ trễ dữ liệu tính bằng giờ hoặc thậm chí bằng ngày.[2] Điều này tạo ra một "khoảng trống thông tin" khiến doanh nghiệp không thể phản ứng kịp thời với các biến động thị trường hoặc sự cố hệ thống.[3] Tầm nhìn của dự án này là thiết lập một "đường ống" dữ liệu (data pipeline) liên tục, nơi dữ liệu giao dịch từ cơ sở dữ liệu lõi (OLTP) được đồng bộ hóa, làm sạch và đưa vào kho phân tích (OLAP) với độ trễ tính bằng giây.[2]

Hệ thống được thiết kế dựa trên nguyên lý tách biệt hoàn toàn giữa lớp ghi dữ liệu và lớp phân tích, sử dụng Apache Kafka làm xương sống để đảm bảo tính đàn hồi và khả năng mở rộng.[5] Việc sử dụng kiến trúc hướng sự kiện (event-driven architecture) cho phép các thành phần hạ nguồn tiêu thụ dữ liệu theo tốc độ riêng của chúng mà không gây ảnh hưởng đến hiệu năng của cơ sở dữ liệu sản xuất.[5]

Chi tiết các Thành phần trong Hệ thống

Hệ thống bao gồm sáu lớp thành phần được tích hợp chặt chẽ, mỗi lớp giải quyết một thách thức cụ thể trong vòng đời của dữ liệu.

Lớp Sinh Dữ liệu và Cơ sở dữ liệu Giao dịch (OLTP)

Thành phần khởi đầu của hệ thống là trình tạo dữ liệu (Data Generator), một kịch bản được viết bằng Python hoặc Java nhằm mô phỏng các hoạt động giao dịch thực tế của doanh nghiệp. Để phản ánh đúng thực tế vận hành, trình tạo này không chỉ chèn dữ liệu hợp lệ mà còn chủ động tạo ra các bản ghi lỗi, chẳng hạn như giao dịch có số tiền âm hoặc trạng thái không xác định,

nhằm kiểm thử khả năng lọc của lớp xử lý.[5]

Cơ sở dữ liệu lõi được đề xuất là PostgreSQL, một hệ quản trị cơ sở dữ liệu quan hệ mạnh mẽ, hỗ trợ cơ chế giải mã logic (logical decoding) cần thiết cho việc thu thập dữ liệu thay đổi.[9]

Lớp Thu thập Dữ liệu Thay đổi (CDC Layer - Debezium)

Debezium đóng vai trò là "người lắng nghe" các thay đổi từ Write-Ahead Log (WAL) của PostgreSQL.[5] Khác với các phương pháp truy vấn định kỳ (polling) vốn gây tải nặng cho database, CDC ghi nhận mọi sự kiện INSERT, UPDATE, DELETE ngay khi chúng được commit mà không làm gián đoạn luồng giao dịch.[9]

Dữ liệu được Debezium bắt giữ sẽ được đóng gói dưới định dạng Avro, một định dạng nhị phân nhỏ gọn giúp tối ưu hóa hiệu suất truyền tải và lưu trữ so với JSON truyền thống.[11] Việc sử dụng Avro kết hợp với Schema Registry cho phép quản lý sự tiến hóa của lược đồ dữ liệu (schema evolution) một cách an toàn, tránh tình trạng lỗi hệ thống khi cấu trúc bảng trong database thay đổi.[11]

Lớp Môi giới Tin nhắn (Message Broker - Kafka KRaft)

Apache Kafka với chế độ KRaft (không phụ thuộc Zookeeper) được chọn làm hệ thống môi giới tin nhắn trung tâm.[7] Kafka cung cấp khả năng lưu trữ bền vững các sự kiện, cho phép nhiều hệ thống tiêu thụ (Flink, Kafka Connect, ClickHouse) cùng đọc dữ liệu từ một nguồn duy nhất mà không xung đột.[5] KRaft giúp đơn giản hóa việc quản trị cụm Kafka và cải thiện tốc độ phục hồi khi có sự cố xảy ra.[7]

Thành phần	Vai trò kỹ thuật	Lợi ích cốt lõi
Kafka Brokers	Lưu trữ và phân phối dòng sự kiện	Đảm bảo tính sẵn sàng cao và khả năng mở rộng [6]
Schema Registry	Quản lý phiên bản của Avro Schemas	Đảm bảo tính tương thích dữ liệu giữa Producer và Consumer [14]
Kafka Connect	Tích hợp nguồn và đích dữ liệu	Tự động hóa việc di chuyển dữ liệu mà không cần code [8]

Lớp Xử lý Luồng (Stream Processing - Apache Flink)

Apache Flink là trái tim của hệ thống xử lý, chịu trách nhiệm biến dữ liệu thô thành thông tin có giá trị.[5] Flink đọc các tin nhắn từ Kafka, bóc tách cấu trúc "envelope" của Debezium (bao gồm

các phần before, after, và op) để trích xuất dữ liệu thực tế.[5]

Quy trình xử lý trong Flink bao gồm:

1. **Lọc (Filter)**: Loại bỏ các bản ghi không hợp lệ theo quy tắc kinh doanh, ví dụ: WHERE amount > 0 AND status = 'SUCCESS'.[5]
2. **Gom nhóm theo cửa sổ (Windowing)**: Sử dụng Tumbling Window để tính toán các chỉ số tổng hợp (như tổng doanh thu mỗi phút theo từng cửa hàng).[5]
3. **Xuất dữ liệu**: Kết quả đã được làm sạch và tổng hợp sẽ được đẩy vào một Kafka Topic mới mang tên clean-transactions.[5]

Lớp Phân tích Trực tuyến (OLAP - ClickHouse)

ClickHouse được lựa chọn làm kho dữ liệu phân tích nhờ tốc độ truy vấn vượt trội đối với các tập dữ liệu quy mô hàng tỷ dòng.[5] ClickHouse sử dụng Kafka Table Engine để tự động tiêu thụ dữ liệu từ topic clean-transactions.[5]

Để đảm bảo tính nhất quán và xử lý các bản ghi trùng lặp (duplicates) phát sinh trong quá trình xử lý luồng, hệ thống sử dụng bảng ReplacingMergeTree kết hợp với Materialized View.[9] Cơ chế này cho phép ClickHouse ghi đè các bản ghi cũ bằng phiên bản mới nhất dựa trên một khóa định danh duy nhất.[9]

Lớp Hồ Dữ liệu (Data Lake - MinIO)

Để phục vụ nhu cầu lưu trữ dài hạn và phân tích hồi cứu (back-testing), toàn bộ dữ liệu thô từ database ban đầu được đẩy vào MinIO thông qua Kafka Connect S3 Sink.[8] Dữ liệu được lưu dưới định dạng Parquet, một định dạng lưu trữ hướng cột giúp tối ưu hóa không gian lưu trữ và tăng tốc độ truy vấn cho các công cụ như Apache Spark hay Trino.[14] Việc phân chia dữ liệu theo cấu trúc thư mục thời gian (year=.../month=...) giúp quản lý hồ dữ liệu một cách khoa học và hiệu quả.[15]

Kế hoạch Triển khai Chi tiết trong 4 Tuần

Dự án được chia thành bốn giai đoạn chính, đi từ việc thiết lập nền tảng hạ tầng đến việc tối ưu hóa và đóng gói vận hành.

Tuần 1: Xây dựng Nền móng OLTP và CDC

Mục tiêu của tuần đầu tiên là thiết lập luồng dữ liệu tự động từ database vào Kafka mà không cần can thiệp vào mã nguồn ứng dụng hiện có.[5]

- **Giai đoạn 1 (Ngày 1-2): Setup Hạ tầng Cơ bản** Hệ thống được khởi tạo thông qua docker-compose.yml, bao gồm PostgreSQL, cụm Kafka KRaft, Schema Registry và dịch vụ Kafka Connect tích hợp plugin Debezium.[5] Việc sử dụng Docker giúp nhất quán môi trường giữa các nhà phát triển và đơn giản hóa quá trình triển khai.
- **Giai đoạn 2 (Ngày 3-4): Data Generator và Cấu hình Postgres** Để CDC hoạt động,

PostgreSQL phải được cấu hình wal_level = logical.[10] Trình tạo dữ liệu bằng Python (sử dụng thư viện Faker) sẽ bắt đầu chèn hàng ngàn bản ghi vào bảng transactions, tạo ra một "dòng chảy" dữ liệu thực tế cho toàn bộ pipeline.[5]

- **Giai đoạn 3 (Ngày 5-7): Triển khai Debezium** Việc cấu hình Debezium được thực hiện qua các lệnh gọi REST API đến Kafka Connect. Ngay khi Connector được kích hoạt, nó sẽ bắt đầu quét toàn bộ dữ liệu hiện có trong bảng (snapshot) và sau đó chuyển sang chế độ theo dõi các thay đổi trực tiếp (streaming).[11]

Tuần 2: Xử lý Luồng Hạng nặng với Apache Flink

Tuần này tập trung vào việc xử lý dữ liệu thô và áp dụng các quy tắc kinh doanh vào dòng sự kiện.[5]

- **Giai đoạn 1 (Ngày 1-2): Setup Flink Cluster** Các thành phần JobManager và TaskManager của Flink được thêm vào hệ sinh thái Docker. Flink JobManager đóng vai trò điều phối các tác vụ, trong khi TaskManager thực hiện việc tính toán thực tế.[5]
- **Giai đoạn 2 (Ngày 3-5): Viết Logic Xử lý bằng Flink SQL** Flink SQL được ưu tiên sử dụng nhờ tính tuyên bố (declarative), giúp các kỹ sư dữ liệu dễ dàng định nghĩa các phép biến đổi mà không cần kiến thức sâu về Java.[1] Một điểm quan trọng là khả năng xử lý "envelope" của Debezium. Flink hỗ trợ format debezium-avro-confluent, giúp nó tự động hiểu rằng dữ liệu cần thiết nằm trong trường after.[17] Các câu lệnh SQL sẽ thực hiện lọc bỏ giao dịch không hợp lệ và thực hiện gom nhóm thời gian (Tumbling Window) một phút để tính toán doanh thu.[5]
- **Giai đoạn 3 (Ngày 6-7): Xuất Dữ liệu ra Kafka** Kết quả từ Flink được ghi vào topic clean-transactions. Đây là một bước quan trọng để tách biệt (decoupling) lớp xử lý và lớp lưu trữ phân tích.[5]

Tuần 3: Tích hợp OLAP với ClickHouse

Tuần này tập trung vào việc đưa dữ liệu đã được xử lý sạch vào môi trường phân tích hiệu năng cao.[5]

- **Giai đoạn 1 (Ngày 1-2): Setup ClickHouse** Server ClickHouse được khởi tạo và cấu hình để sẵn sàng nhận dữ liệu. ClickHouse được thiết kế để xử lý ghi theo lô (bulk insert) hơn là ghi lẻ tẻ, điều này rất phù hợp với luồng dữ liệu từ Kafka.[5]
- **Giai đoạn 2 (Ngày 3-5): Cấu hình Kafka Engine** ClickHouse sử dụng một loại bảng đặc biệt là ENGINE = Kafka để tự động đăng ký (subscribe) vào topic clean-transactions.[5] Bảng này đóng vai trò như một buffer, tạm thời giữ dữ liệu trước khi đẩy vào kho lưu trữ chính.
- **Giai đoạn 3 (Ngày 6-7): Xử lý Upsert với Materialized View** Do Flink có thể gửi lại dữ liệu (retries) hoặc cập nhật kết quả trong cùng một cửa sổ thời gian, việc sử dụng ReplacingMergeTree là bắt buộc để tránh tính toán sai lệch do trùng lặp dữ liệu.[9] Một Materialized View sẽ được tạo ra để tự động di chuyển dữ liệu từ bảng Kafka Engine sang

bảng ReplacingMergeTree.[5]

Tuần 4: Hoàn thiện Data Lake MinIO và Đóng gói Dự án

Giai đoạn cuối cùng nhằm đảm bảo tính bền vững của dữ liệu và khả năng bàn giao dự án.[5]

- **Giai đoạn 1 (Ngày 1-2): Setup MinIO và Kafka Connect S3 Sink** MinIO được cấu hình như một hệ thống lưu trữ đối tượng tương thích S3. Kafka Connect S3 Sink sẽ đọc từ topic thô ban đầu để đảm bảo không một bit dữ liệu nào bị mất mát.[8] Dữ liệu được ghi dưới dạng file Parquet phân chia theo cấu trúc thời gian để phục vụ các mục đích kiểm toán hoặc phân tích chuyên sâu sau này.[15]
- **Giai đoạn 2 (Ngày 3-5): Tối ưu hóa và Xử lý Lỗi** Hệ thống được thử thách dưới tải trọng lớn (1000 records/s) để đo lường độ trễ và khả năng chịu tải của Flink cũng như ClickHouse.[3] Các kịch bản lỗi như mất kết nối mạng hoặc server chết được giả lập để kiểm tra cơ chế Checkpointing của Flink và khả năng lưu giữ offset của Kafka.[5]
- **Giai đoạn 3 (Ngày 6-7): Viết Tài liệu (Documentation)** Đây là bước then chốt để đảm bảo dự án có thể vận hành bởi các đội ngũ khác. Tài liệu bao gồm sơ đồ kiến trúc, hướng dẫn thiết lập từng bước (step-by-step) và giải thích các quyết định thiết kế cốt lõi.[27]

Phân tích Chuyên sâu về Cơ chế Hoạt động và Tối ưu hóa

Sự thành công của một hệ thống dữ liệu real-time không chỉ nằm ở việc chọn đúng công cụ mà còn ở việc hiểu rõ cách chúng tương tác và các cạm bẫy kỹ thuật có thể xảy ra.

Tại sao sử dụng Debezium và CDC thay vì Application-level Events?

Một lỗi phổ biến trong kỹ thuật dữ liệu là để ứng dụng tự gửi sự kiện vào Kafka ngay khi ghi vào database (Dual Write). Cách tiếp cận này rất dễ dẫn đến tình trạng không nhất quán dữ liệu nếu một trong hai thao tác thất bại.[11] CDC giải quyết triệt để vấn đề này bằng cách sử dụng chính bản ghi thay đổi của database làm nguồn sự thật duy nhất, đảm bảo tính nhất quán tuyệt đối giữa database sản xuất và các hệ thống hạ nguồn.[11]

Tối ưu hóa Hiệu suất ClickHouse thông qua Kafka Engine

Việc sử dụng Kafka Engine trong ClickHouse giúp loại bỏ nhu cầu về các công cụ trung gian như Kafka Connect Sink cho ClickHouse, từ đó giảm độ phức tạp và chi phí vận hành.[5]

Tham số cấu hình	Tác động đến hiệu năng	Khuyến nghị
kafka_max_block_size	Kiểm soát kích thước mỗi lần flush dữ liệu	Tăng lên (500k-1M) để tối ưu hóa việc ghi lô [20]

kafka_num_consumers	Số lượng luồng đọc song song từ Kafka	Đặt bằng số lượng partition của topic để đạt throughput tối đa [20]
stream_flush_interval_ms	Thời gian tối đa giữa các lần flush	Điều chỉnh dựa trên yêu cầu về độ trễ (1-10 giây) [20]

Chiến lược Xử lý Duplicate với ReplacingMergeTree

Trong ClickHouse, việc xóa dữ liệu hoặc cập nhật bản ghi là các thao tác tốn kém. ReplacingMergeTree giải quyết vấn đề này bằng cách thực hiện việc "xóa" bản ghi cũ và giữ bản ghi mới trong quá trình merge các phần dữ liệu (data parts) ở nền (background).[9] Tuy nhiên, cần lưu ý rằng cơ chế này là "cuối cùng nhất quán" (eventual consistency). Để lấy được dữ liệu chính xác ngay lập tức tại thời điểm truy vấn, người dùng nên sử dụng từ khóa FINAL hoặc thực hiện GROUP BY trên khóa chính.[22]

Quản lý Chất lượng và Giám sát Hệ thống

Một hệ thống dữ liệu thời gian thực nếu không được giám sát chặt chẽ sẽ nhanh chóng trở nên không đáng tin cậy. Việc thiết lập các chỉ số SLIs (Service Level Indicators) và SLOs (Service Level Objectives) là bắt buộc trong môi trường doanh nghiệp.[1]

Các Chỉ số Giám sát Then chốt

Hệ thống sử dụng Prometheus để thu thập số liệu và Grafana để hiển thị dashboard. Các chỉ số cần được theo dõi sát sao bao gồm:

1. **Consumer Lag (Kafka)**: Khoảng cách giữa tin nhắn mới nhất trong Kafka và tin nhắn mà Flink hoặc ClickHouse đang xử lý.[15] Lag cao là dấu hiệu cho thấy hệ thống xử lý đang bị quá tải hoặc gặp lỗi.
2. **Milliseconds Behind Source (Debezium)**: Thời gian chênh lệch giữa lúc giao dịch được commit trong Postgres và lúc Debezium bắt được nó.[25] Chỉ số này đo lường độ tươi của dữ liệu (data freshness).
3. **Checkpoint Duration (Flink)**: Thời gian Flink thực hiện sao lưu trạng thái. Nếu thời gian này tăng đột biến, nó có thể gây ra hiện tượng "back-pressure", làm chậm toàn bộ pipeline.[7]
4. **Merge Performance (ClickHouse)**: Tỷ lệ dữ liệu chưa được merge trong ClickHouse. Nếu dữ liệu trùng lặp tồn tại quá lâu, nó sẽ ảnh hưởng đến tốc độ truy vấn phân tích.[21]

Quản lý Lỗi và Khả năng Phục hồi

Hệ thống được thiết kế với tư duy "fail-fast and recover-fast".

- **Flink Checkpointing**: Định kỳ lưu trữ trạng thái của luồng xử lý vào một bộ lưu trữ bền vững. Nếu một node Flink bị lỗi, nó có thể phục hồi từ checkpoint gần nhất và tiếp tục xử

lý với cam kết exactly-once.[5]

- **Kafka Persistence**: Kafka lưu giữ dữ liệu trên đĩa trong một khoảng thời gian cấu hình (retention period), cho phép các hệ thống hạ nguồn "đọc lại" dữ liệu nếu gặp sự cố nghiêm trọng.[5]

Bảo mật và Tuân thủ Dữ liệu

Trong môi trường doanh nghiệp, dữ liệu giao dịch thường chứa thông tin nhạy cảm. Do đó, bảo mật phải được tích hợp ở mọi tầng của kiến trúc.[32]

Kiểm soát Truy cập và Phân quyền (RBAC)

ClickHouse và Kafka đều được cấu hình với mô hình RBAC (Role-Based Access Control).

- Các ứng dụng dashboard chỉ được cấp quyền SELECT trên các bảng phân tích.[32]
- Dịch vụ Kafka Connect chỉ được cấp quyền READ từ các topic nguồn và WRITE vào các topic đích tương ứng.[19]

Mã hóa Dữ liệu

Tất cả các kết nối giữa các thành phần (Flink tới Kafka, Kafka tới ClickHouse) phải được thực hiện qua giao thức mã hóa TLS 1.2+.[32] Đối với dữ liệu lưu trữ tại MinIO, có thể kích hoạt cơ chế mã hóa ở trạng thái nghỉ (encryption at rest) để đảm bảo an toàn nếu hạ tầng vật lý bị xâm nhập.[32]

Tối ưu hóa Lưu trữ Hồ Dữ liệu trên MinIO

Vấn đề thường gặp nhất khi sử dụng S3 Sink là hiện tượng "Small Files Problem" – hàng ngàn file nhỏ được tạo ra gây chậm hệ thống file. Để giải quyết vấn đề này, các tham số của S3 Sink Connector cần được điều chỉnh:

- **flush.size**: Số lượng bản ghi tối thiểu trước khi đóng một file Parquet. Giá trị khuyến nghị là từ 10.000 đến 100.000 tùy thuộc vào kích thước bản ghi.[15]
- **rotate.interval.ms**: Thời gian tối đa để mở một file. Điều này đảm bảo rằng ngay cả khi dữ liệu đến chậm, file vẫn được đóng định kỳ để dữ liệu có thể được truy cập bởi các công cụ khác.[15]

Định dạng	Ưu điểm chính	Sử dụng phù hợp
Avro	Có schema đi kèm, nén tốt	Truyền tải dữ liệu giữa các thành phần real-time [11]

Parquet	Lưu trữ hướng cột, cực nhanh cho analytics	Lưu trữ dài hạn trong Data Lake (MinIO) [14]
JSON	Dễ đọc, linh hoạt	Debug và phát triển ban đầu [14]

Kết luận và Khuyến nghị

Kiến trúc hệ thống dữ liệu real-time tích hợp Debezium, Kafka, Flink và ClickHouse là một giải pháp toàn diện cho những doanh nghiệp đang tìm kiếm sự tức thời và độ chính xác trong phân tích dữ liệu.[5] Bằng cách tách biệt lớp giao dịch và lớp phân tích thông qua một pipeline sự kiện mạnh mẽ, hệ thống không chỉ giải quyết được bài toán về độ trễ mà còn đảm bảo khả năng mở rộng không giới hạn khi khối lượng dữ liệu tăng trưởng.

Sự thành công của dự án phụ thuộc lớn vào việc tuân thủ các quy trình triển khai nghiêm ngặt trong 4 tuần, đặc biệt là các giai đoạn tối ưu hóa và giám sát. Việc hiểu sâu các cơ chế nội tại như ReplacingMergeTree trong ClickHouse hay Checkpointing trong Flink sẽ giúp các kỹ sư dữ liệu xây dựng được một hệ thống không chỉ nhanh mà còn cực kỳ bền bỉ và tin cậy. Trong tương lai, hệ thống này có thể dễ dàng mở rộng để tích hợp các mô hình Machine Learning trực tiếp vào luồng xử lý, mang lại khả năng phân tích dự báo (predictive analytics) vượt trội cho doanh nghiệp.[1]

Works cited

1. Real-Time Data Pipelines for Building Instant Insights | Integrate.io, accessed April 6, 2026, https://www.integrate.io/blog/real-time-data-pipelines-building-instant-insights/
2. Streaming Data Pipelines - Confluent, accessed April 6, 2026, https://www.confluent.io/learn/streaming-data-pipelines/
3. Building a Real-Time Data Pipeline for Enhanced Analytics - Curate Partners, accessed April 6, 2026, https://curatepartners.com/case-study/real-time-data-pipeline-enhanced-analytics/
4. 15 Data Pipeline Examples (Including Real Life Mega Pipelines) - Dagster, accessed April 6, 2026, https://dagster.io/learn/data-pipelines-examples
5. Building a Real-Time Order Analytics Pipeline with Kafka, Apache ..., accessed April 6, 2026, https://medium.com/@sateeshpy/building-a-real-time-order-analytics-pipeline-with-kafka-apache-flink-and-clickhouse-f1be337f7a71
6. The KFC Architecture Blueprint: Kafka, Flink, and ClickHouse - BigData Boutique, accessed April 6, 2026, https://bigdataboutique.com/blog/kfc-architecture-blueprint-kafka-flink-and-clickhouse
7. Building a Real-Time Order Analytics Pipeline with Kafka, Apache Flink, and

ClickHouse, accessed April 6, 2026, https://sateeshpy.medium.com/building-a-real-time-order-analytics-pipeline-with-kafka-apache-flink-and-clickhouse-f1be337f7a71?source=rss-------1
8. The simplest way to make Kafka MinIO work like it should - Hoop.dev, accessed April 6, 2026, https://hoop.dev/blog/the-simplest-way-to-make-kafka-minio-work-like-it-should
9. Deduplication strategies (using CDC) | ClickHouse Docs, accessed April 6, 2026, https://clickhouse.com/docs/integrations/clickpipes/postgres/deduplication
10. How to Implement Change Data Capture with Debezium and Kafka - OneUptime, accessed April 6, 2026, https://oneuptime.com/blog/post/2026-02-20-cdc-debezium-kafka/view
11. Implementing CDC with Debezium - Conduktor, accessed April 6, 2026, https://www.conduktor.io/glossary/implementing-cdc-with-debezium
12. Why Engine Choice Matters for CDC in ClickHouse: MergeTree vs Replacing vs Collapsing Explained - Fiveonefour, accessed April 6, 2026, https://www.fiveonefour.com/blog/clickhouse-table-engines-and-cdc
13. Connect Streamkap to ClickHouse, accessed April 6, 2026, https://clickhouse.com/docs/integrations/sttreamkap
14. Parquet Output From Kafka Connect to S3 - Codemia, accessed April 6, 2026, https://codemia.io/knowledge-hub/path/parquet_output_from_kafka_connect_to_s3
15. How to Sink Kafka Data to S3 - OneUptime, accessed April 6, 2026, https://oneuptime.com/blog/post/2026-01-21-kafka-connect-s3-sink/view
16. Schema registries for Kafka ClickPipe | ClickHouse Docs, accessed April 6, 2026, https://clickhouse.com/docs/integrations/clickpipes/kafka/schema-registries
17. Debezium-JSON - BytePlus, accessed April 6, 2026, https://docs.byteplus.com/en/docs/flink/Debezium-JSON
18. Monitoring and Analytics Stack with Prometheus, Grafana, ClickHouse, Kafka, and Slack Alerts | by SarahW | Medium, accessed April 6, 2026, https://medium.com/@sarahwang9/monitoring-and-analytics-stack-with-prometheus-grafana-clickhouse-kafka-and-slack-alerts-df5858e620c9
19. The simplest way to make ClickHouse Kafka work like it should - Hoop.dev, accessed April 6, 2026, https://hoop.dev/blog/the-simplest-way-to-make-clickhouse-kafka-work-like-it-should
20. Using the Kafka table engine | ClickHouse Docs, accessed April 6, 2026, https://clickhouse.com/docs/integrations/kafka/kafka-table-engine
21. How to Monitor Kafka with ClickHouse® Kafka Engine - Tinybird, accessed April 6, 2026, https://www.tinybird.co/blog/kafka-monitoring-clickhouse-kafka-engine
22. ReplacingMergeTree table engine | ClickHouse Docs, accessed April 6, 2026, https://clickhouse.com/docs/engines/table-engines/mergetree-family/replacingmergetree
23. How to Choose the Right ClickHouse Table Engine - OneUptime, accessed April 6, 2026,

https://oneuptime.com/blog/post/2026-01-21-clickhouse-table-engine-selection/view
24. Looking for cold storage architecture advice: Geospatial time series data from Kafka → S3/MinIO : r/dataengineering - Reddit, accessed April 6, 2026, https://www.reddit.com/r/dataengineering/comments/1pbyg9g/looking_for_cold_storage_architecture_advice/
25. How to Monitor Debezium Connectors - OneUptime, accessed April 6, 2026, https://oneuptime.com/blog/post/2026-01-28-monitor-debezium-connectors/view
26. Debezium Avro Confluent - Ververica documentation, accessed April 6, 2026, https://docs.ververica.com/managed-service/reference/formats/debezium-avro-confluent/
27. Software requirement document template: free SRS steps [2026] - Asana, accessed April 6, 2026, https://asana.com/resources/software-requirement-document-template
28. Product Requirements Document (PRD) Template + Examples - Studio Red, accessed April 6, 2026, https://www.studiored.com/blog/eng/product-requirements-document-template/
29. Deduplication strategies | ClickHouse Docs, accessed April 6, 2026, https://clickhouse.com/docs/guides/developer/deduplication
30. accessed January 1, 1970, https://nightlies.apache.org/flink/flink-docs-stable/docs/ops/metrics/
31. How to monitor Clickhouse with Grafana Cloud, accessed April 6, 2026, https://grafana.com/blog/how-to-start-monitoring-your-clickhouse-instance-or-cluster-with-grafana-cloud/
32. ClickHouse Security Hardening Checklist - OneUptime, accessed April 6, 2026, https://oneuptime.com/blog/post/2026-03-31-clickhouse-security-hardening-checklist/view
33. How to Harden ClickHouse Server for Production - OneUptime, accessed April 6, 2026, https://oneuptime.com/blog/post/2026-03-31-clickhouse-harden-server-production/view
34. How to Secure a Flink Job with Confluent Manager for Apache Flink, accessed April 6, 2026, https://docs.confluent.io/cp-flink/current/jobs/configure/security.html
35. How to Manage ClickHouse Users and Access Control - OneUptime, accessed April 6, 2026, https://oneuptime.com/blog/post/2026-01-21-clickhouse-users-access-control/view
36. Three Kafka S3 Sink Settings for Easier File Processing | by ..., accessed April 6, 2026, https://medium.com/binary-notes/three-kafka-s3-sink-settings-for-easier-file-processing-419eab09c903